

Guía de uso de la librería bcryptjs en Node.js

1. Instalación de bcryptjs

```
npm install bcryptjs
```

2. Uso básico de bcryptjs

La librería bcryptjs es muy útil para trabajar con contraseñas, ya que permite **encriptar** contraseñas y verificar si una contraseña ingresada corresponde con una almacenada de forma segura.

A continuación, se describen dos funciones comunes que puedes utilizar con bcryptjs: **generar el hash de una contraseña y comparar una contraseña ingresada con el hash almacenado.**

3. Generar un hash para la contraseña

Cuando un usuario se registra en el sistema, la contraseña debe ser cifrada antes de almacenarla. En el siguiente código, en el endpoint userPost, se puede ver cómo se genera un hash para la contraseña y se guarda en la base de datos.

```
userPost: async (req, res) => {  
  
  let { email, name, documenttype, document, password } = req.body;  
  name = name.toUpperCase();  
  const user = new User({ email, name, documenttype, document });  
  
  const salt = bcryptjs.genSaltSync();  
  user.password = bcryptjs.hashSync(password, salt)  
  await user.save()  
  res.json({  
    deviceid: user.deviceid,  
    certificatepath: `certificado${document}.png`,  
    document: user.document  
  })  
},
```

Un **hash** es una función matemática que convierte una entrada (en este caso, una contraseña) en una cadena de longitud fija, que parece aleatoria. El resultado del hash es único para cada entrada, lo que significa que incluso un cambio pequeño en la entrada resultará en un valor hash completamente diferente.

En el contexto de las contraseñas, el hash se utiliza para almacenar la contraseña de forma segura. En lugar de guardar la contraseña original en la base de datos, se guarda su versión encriptada (el hash). De esta forma, si un atacante accede a la base de datos, no podrá ver las contraseñas originales.

Por ejemplo, si tienes una contraseña como "miContraseña123", al aplicarle un algoritmo hash como bcrypt, obtendrás un valor como:

\$2a\$10\$zjZYJjxPZ7nSgX5e4JfpheEgZckKm4FFU54Wy28omWVA78gN2vNiS

Mirando nuevamente el ejemplo de cómo generar un hash para la contraseña:

```
const bcryptjs = require('bcryptjs');

let password = 'contraseñaSegura123'; // La contraseña ingresada
por el usuario

// Generar un "salt" que se utiliza para mejorar la seguridad
const salt = bcryptjs.genSaltSync();

// Crear el hash de la contraseña
const hashedPassword = bcryptjs.hashSync(password, salt);

// Aquí se guardaría el "hashedPassword" en la base de datos.
console.log(hashedPassword);
```

1. **Obtenemos los datos** de la solicitud (email, name, documenttype, document, password).
2. **Generamos el "salt"** con bcryptjs.genSaltSync():

Un **salt** es un valor aleatorio que se utiliza junto con la contraseña original para crear el hash. El propósito del salt es evitar ataques de "rainbow table" (tablas precomputadas de hashes) que intentan adivinar las contraseñas mediante la comparación de hashes previamente generados.

Cuando usas un salt, cada vez que se genera un hash de una contraseña, incluso si dos usuarios tienen la misma contraseña, los hashes serán diferentes debido a que el salt es único para cada uno. Esto aumenta significativamente la seguridad de las contraseñas almacenadas.

Por ejemplo, si la contraseña "miContraseña123" se encripta con dos salts diferentes, se obtendrán dos hashes completamente distintos

3. El "salt" es un valor aleatorio que se utiliza para hacer más segura la encriptación.
4. **Generamos el hash** de la contraseña usando bcryptjs.hashSync(password, salt).
5. **Guardamos** el hash de la contraseña en la base de datos, en lugar de la contraseña original.

4. Verificar si la contraseña es correcta

Cuando un usuario intenta iniciar sesión, necesitamos comparar la contraseña ingresada con el hash almacenado en la base de datos. Para ello, usamos `bcryptjs.compareSync()`.

Así se vería nuestro endpoint de login.

```
login: async (req, res) => {
  const { email, password } = req.body;
  try {
    const user = await User.findOne({ email })

    if (!user) {
      return res.status(400).json({
        msg: "Usuario / Password no son correctos!"
      })
    }

    if (user.estado === 0 || user.rol !== "Admin") {
      return res.status(400).json({
        msg: "Usuario Inactivo"
      })
    }

    const validPassword = bcryptjs.compareSync(password, user.password);
    if (!validPassword) {
      return res.status(400).json({
        msg: "Usuario / Password no son correctos"
      })
    }

    const token = await generarJWT(user.id);

    res.json({
      id: user._id,
      token
    })
  } catch (error) {
    return res.status(500).json({
      msg: "Hable con el WebMaster"
    })
  }
},
```

El paso a paso:

```
let passwordIngresada = 'contraseñaSegura123'; // La
contraseña que el usuario ingresa en el login

let hashedPasswordAlmacenada =
'$2a$10$zjZYJjxPZ7nSgX5e4JfpheEgZckKm4FFU54Wy28omW78gN2vNiS';
// El hash de la contraseña almacenado en la base de datos
```

```
// Comparamos la contraseña ingresada con el hash almacenado
const esValido = bcryptjs.compareSync(passwordIngresada,
hashedPasswordAlmacenada);

if (esValido) {
    console.log("Contraseña válida");
} else {
    console.log("Contraseña inválida");
}
```

En el endpoint login, el flujo es el siguiente:

1. **Recibimos los datos** de la solicitud (email, password).
2. **Buscamos al usuario** en la base de datos usando el email.
3. **Comprobamos el estado y el rol** (y todo lo que necesites validar, pagos, etc) del usuario para asegurarnos de que esté activo y sea un administrador.
4. **Comparamos la contraseña ingresada** con el hash almacenado en la base de datos usando `bcryptjs.compareSync(password, user.password)`.
5. Si las contraseñas coinciden, **generamos un token JWT** para autenticar al usuario.

5. Resumen de los métodos más utilizados en bcryptjs

- **bcryptjs.genSaltSync()**: Genera un "salt" de forma síncrona, que es un valor aleatorio utilizado para mejorar la seguridad del hash.
- **bcryptjs.hashSync(password, salt)**: Encripta la contraseña usando un "salt" y devuelve el hash resultante.
- **bcryptjs.compareSync(password, hash)**: Compara una contraseña proporcionada con un hash almacenado y devuelve true si coinciden.

6. Consideraciones adicionales

- **Seguridad**: Siempre debes almacenar las contraseñas de tus usuarios como hashes, nunca las contraseñas en texto plano. El uso de bcryptjs para este propósito es una buena práctica, ya que su algoritmo está diseñado específicamente para hacer que los ataques de fuerza bruta sean más lentos y difíciles.